Ensembling Diverse Collaborative Filtering Models for Improved Recommendations

Giacomo Fiorindo [*1] **Adam Mernissi Arifi** [*1] **Mehdi Ziazi** [*1] **Pierre-Louis Bravais** [*1]

Abstract

Collaborative filtering is a method used in recommendation systems to predict the preferences of users by leveraging the preferences of many other users. The role of collaborative filtering in recommendation systems is pivotal as it helps in personalizing user experiences without needing extensive content analysis. In this paper we explore several collaborative filtering techniques in the context of the CIL 2024 Kaggle competition, ranging from matrix factorization methods to neural-network-based approaches. We propose a blending architectures that achieves a score of 0.96677 in the public test.

1. Introduction

In today's digital age, the sheer volume of available information presents both opportunities and challenges for users seeking personalized experiences. Collaborative filtering (CF) has become a pivotal technique in recommendation systems, offering tailored suggestions by leveraging the preferences and behaviors of a community of users, by assuming that if a person A has the same opinion as a person B on an issue, A is more likely to have B's opinion on a different issue than that of a randomly chosen person.

Collaborative filtering originated in the mid-1990s (Goldberg et al., 1992; Resnick et al., 1994) with early recommendation systems and has significantly evolved with the rise of large-scale internet platforms. Companies like Amazon (Smith & Linden, 2017) and Netflix (Bennett et al., 2007) have successfully used CF algorithms to drive user engagement and satisfaction by predicting users' interests based on the preferences of similar users. These systems are critical not only for improving user experience but also for increasing retention rates and driving commercial success.

Despite its widespread use, collaborative filtering faces several challenges. Issues such as data sparsity, scalability, and the cold start problem—where new users or items lack sufficient interaction data—pose significant hurdles. Additionally, ensuring diversity and avoiding the filter bubble effect, where users are only exposed to information that aligns with their existing preferences, remains an ongoing concern.

This paper delves into the intricacies of collaborative filtering, exploring its methodologies and challenges in the context of the CIL 2024 Kaggle competition. Finally, we propose an ensemble of the aforementioned techniques to generate the predictions.

2. Preliminaries

In our setting, we are provided with a dataset containing $1,176,952$ ratings, from 1 to 5, given by $n = 10,000$ users for $m = 1,000$ different items. In the paper, we use the following notation:

- $I = [n]$ indicates the set of users.
- $J = [m]$ indicates the set of items.
- $\Omega \subset I \times J$ indicates the set user-item pairs with an observed rating. Note that $|\Omega| = 1,176,952$.
- $A \in \{0, 1, ..., 5\}^{I \times J}$ indicates the user-item interaction matrix containing the given ratings. The value of missing entries depends on the specific method used.

Our task is to build a collaborative filtering system to predict the missing values in the matrix A. We measure the quality of our approach using the Root Mean Squared Error (RMSE), defined as

$$RMSE = \sqrt{\frac{1}{|\Omega|} \sum_{(i,j) \in \Omega} (A_{ij} - \bar{A}_{ij})^2}$$

where $\bar{A}_{ij}$ indicates the predicted rating by user i for item j.

[*]Equal contribution [1]Department of Computer Science, ETH Zurich, Zurich, Switzerland.

Computational Intelligence Lab FS2024, ETH Zuerich

3. Models and Methods

3.1. Matrix Factorization

Matrix factorization is a class of collaborative filtering algorithms used to uncover the latent factors that explain observed interactions between users and items. These techniques work by decomposing the user-item interaction matrix into the product of lower-dimensional matrices, revealing the underlying structure in the data. Singular Value Decomposition (SVD), its enhanced version SVD++, Alternating Least Squares (ALS) and Bayesian Matrix Factorization (BMF) are prominent examples of matrix factorization methods that have been successfully applied in recommendation systems to improve prediction accuracy and scalability.

3.1.1. SVD

SVD aims to approximate the user-item interaction matrix A as the product of three lower-dimensional matrices $U \in \mathbb{R}^{n\times k}$, $\Sigma \in \mathbb{R}^{k\times k}$ and $V \in \mathbb{R}^{k\times m}$ for some $k \leq \min(n, m)$, where Σ is a diagonal matrix and U and V represent latent factors for users and items respectively. The missing rating $\hat{A}_{ij}$ can then be estimated as the inner product $\hat{A}_{ij} = u_i^T v_j$ of the user factor u_i and the item factor v_j.

To prevent overfitting, SVD includes regularization terms and optimizes the following loss function:

$$\ell(U, V) = \frac{1}{2} \sum_{(i,j)\in\Omega} (A_{ij} - u_i^T v_j)^2 + \lambda(\|U\|_F^2 + \|V\|_F^2)$$

Here, Ω represents the set of observed user-item pairs, and $\lambda > 0$ is a regularization hyperparameter. SVD minimizes this loss function using stochastic gradient descent, adjusting the user and item factors iteratively. SVD is the first of our baselines.

3.1.2. SVD++

SVD++ (Koren, 2008) extends SVD by incorporating implicit feedback, assuming that a user's interaction with an item, even without an explicit rating, indicates a preference. This model modifies the rating estimate to account for implicit interactions:

$$\hat{A}_{ij} = \bar{A} + b_{ui} + b_{ij} + v_j^T \left(u_i + |I(u)|^{-1/2} \sum_{j'\in I(u)} p_{j'} \right)$$

where $\bar{R}$ represents the global mean rating, b_{ui} and b_{ij} are the user and item biases respectively, $I(u)$ is the set of items that user u has interacted with and $p_{j'}$ are additional learned item interaction factors.

The term $|I(u)|^{-1/2} \sum_{j'\in I(u)} p_{j'}$ adjusts the user factor u_i with the implicit feedback, improving the accuracy of the model by considering both explicit and implicit user interactions.

3.1.3. Alternating Least Squares (ALS)

Alternating Least Squares (ALS) is a prominent matrix factorization technique used in collaborative filtering, particularly effective for large-scale recommendation systems. Similarly to SVD, the core idea of ALS is to factorize the user-item interaction matrix A into two lower-dimensional matrices $U \in \mathbb{R}^{n\times k}$ and $V \in \mathbb{R}^{k\times m}$ for some $k \leq \min(n, m)$, representing latent factors for users and items respectively, such that $A \approx UV$. By doing so, ALS captures the underlying structure in user preferences and item characteristics, facilitating the prediction of missing entries in the interaction matrix.

ALS's learning process alternates between fixing one matrix and solving for the other, minimizing the same objective function as SVD.

In our experiments, initializing missing values A_{ij} as 0 and using parameters $k = 7$ and $\lambda = 0.08$ for 200 iterations yielded the best results.

3.1.4. Bayesian Factorization Machines (BFM)

Bayesian Factorization Machines (BFMs) improve upon traditional Factorization Machines (FMs) by utilizing Bayesian inference, which removes the necessity for manual hyperparameter tuning and often surpasses traditional optimization methods thanks to Gibbs sampling, a Markov Chain Monte Carlo (MCMC) method that efficiently performs posterior inference.

Feature engineering is a major strength of FMs, as they can effectively handle both explicit and implicit feedback. Explicit feedback, such as ratings, is combined with feature vectors created through one-hot encoding of users and items, (u, i). Meanwhile, implicit feedback—gathered from user behavior rather than direct input—provides additional insight. For example, this can be constructed by identifying all movies a user has rated, forming a feature vector (u, i, i_u) similar to SVD++, where i_u represents implicit user feedback. On the other hand, examining which users have rated a specific movie creates a feature vector (u, i, i_i), where i_i indicates implicit item feedback.

There are two main approaches to incorporate latent factors and biases while capturing diverse interactions within the data and enhance predictive accuracy: Regression and Ordered Probit (Rendle et al., 2019).

The BFM Regression Model uses a standard regression framework to predict user ratings, minimizing prediction error between actual and predicted ratings. It's straightforward and computationally efficient but may not effectively

capture the ordinal nature of rating data, even when leveraging latent factors and biases or implicit features to account for broader interaction contexts.

The Ordered Probit Model is designed for ordinal data, modeling the probability of each rating category separately. It transforms ratings to a zero-based scale and uses a probit function to predict the probabilities of each rating category.

Evaluating these models revealed that the Ordered Probit approach better captures the ordinal nature of ratings, demonstrating its superior performance in collaborative filtering tasks.

To further enhance prediction accuracy, we extended both architectures with a two-step approach. First, we use a BMF mdoel to predict ratings and calculate residuals. Then, we feed these residuals to a LightGBM model (Ke et al., 2017), which refines the initial predictions using user and item indices as features. By combining BMF's strengths in bias and latent factor modeling with LightGBM's ability to adjust residuals, this hybrid approach outperformed the base BFM models.

3.2. NCF

Neural Collaborative Filtering (NCF) is an approach in recommender systems that leverages neural networks to predict user-item interactions. Due to their capability to approximate any function, neural networks are interesting candidates for matrix completion problems. At its core, NCF uses embedding layers to convert users and items into dense vector representations, capturing their latent features. These embeddings are learned from data, allowing the model to understand complex patterns in user preferences and item characteristics. The embeddings for users and items are concatenated to form a combined vector, representing a specific user-item pair. This combined vector is then fed into a series of linear layers, often with activation functions like ReLU, to learn the interaction between users and items. Each linear layer progressively refines the combined representation by either reducing or expanding its dimensionality, ultimately producing a single output value that represents the predicted interaction score. This architecture enables NCF to model both linear and non-linear relationships for generating recommendations.

In the original approach to NCF,which we take as our second baseline, look-up tables are used for embeddings and the dot product of the two embeddings is used as input to the predictor, which we believe is limiting. After several tests, we independently developed a model that closely resembles J-NCF$_c$ (Chen et al., 2019). This model employs two Multilayer Perceptrons to generate the embeddings and uses the concatenation of these embeddings as input to the final Perceptron. With these modifications, we observed significant improvements over the original approach. Our final architecture uses two 4-layer MLPs for embeddings ($[d - 512 - 320 - 128]$), each followed by a Dropout layer with a probability of 0.1, and a 3-layer MLP for the predictor ($[256 - 64 - 1]$). Scaled Exponential Linear Unit (SELU) is used as activation function.

3.3. Autoencoders

Autoencoders are a type of neural network used for unsupervised learning, particularly well-suited for tasks such as dimensionality reduction, feature learning, and anomaly detection. In the context of recommendation systems, autoencoders can effectively learn latent representations of users and items, enabling the prediction of missing entries in user-item interaction matrices (Sedhain et al., 2015; Strub & Mary, 2015).

An autoencoder consists of two main components: an encoder $E : \mathbb{R}^d \to \mathbb{R}^k$ and a decoder $D : \mathbb{R}^k \to \mathbb{R}^d$. The encoder maps the input data into a latent space of lower dimensions, while the decoder attempts to reconstruct the original input from this latent representation. Inspired by the work done at NVIDIA (Kuchaiev & Ginsburg, 2017), both E and D are deep Multilayer Perceptrons with SELU as source of non-linearity. and a Dropout layer between them for regularisation. Contrary to what is stated in the paper and generally believed by the literature (Sarwar et al., 2001), an user-based collaborative approach was found to be better, i.e. $d = n$. We believe the reason for this divergence lies in the training data, which does not fully resemble real-world datasets, as they usually have much higher sparsity.

The network is trained to minimize the reconstruction error using the mean squared error (MSE) with unobserved values zeroed as loss function:

$$MSE = \frac{1}{m} \sum_{i,j \in \Omega} (A_{ij}^T - D(E(A_i^T))_j)^2$$

Our model combines a 3-layers encoder ($[m - 512 - 320 - 128]$) and a 3-layers decoder ($[128 - 320 - 512 - m]$) with a latent space of dimension $k = 128$, a dropout probability of 0.1 and no re-feeding. In addition to the DeepRec structure, we found beneficial to add learnable user bias and movie bias vectors.

3.4. LightGCN

The LightGCN model (He et al., 2020), is an embedding-based model. It builds upon existing GCN (Graph Convolutional Network) models since it's based on neighborhood aggregation. LightGCN achieves more simplicity, scalability and efficiency than traditional GCN models by removing feature transformation and nonlinear activation functions. It

is an embedding-based model, which means it aims to discover the best embeddings for users and items. It learns user and item embeddings by linearly propagating them across the user-item interaction graph, using this graph convolution operation (propagation rule):

For users (the formula is reciprocal for items) :

$$\mathbf{e}_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u|}\sqrt{|\mathcal{N}_i|}} \mathbf{e}_i^{(k)}$$

After multiple propagation layers, the final embedding of a user or item is obtained by combining the embeddings from all layers. The combination rule is a weighted sum:

For users: For items:

$$\mathbf{e}_u = \sum_{k=0}^{K} \alpha_k \mathbf{e}_u^{(k)} \qquad \mathbf{e}_i = \sum_{k=0}^{K} \alpha_k \mathbf{e}_i^{(k)}$$

Finally, the final score for a user-item pair is computed as the inner product of their embeddings.

In our experiment, we used an Adam optimizer as suggested in the paper for model training, but we replaced the loss with RMSE. To prevent overfitting, we added L2 Regularization and early stopping.

3.5. Experiments

Inspired by the positive results of autoencoders and neural collaborative filtering, we developed a model that integrates both approaches. Although this resulted in a model similar to Collaborative Filtering with Dual Autoencoder (CFDA) (Liu & Wang, 2022), it did not achieve satisfactory performance and we decided not to investigate it further.

We also explored a Neural Collaborative Filtering (NCF) approach combining Generalized Matrix Factorization (GMF) with a Multi-Layer Perceptron (MLP) to predict user-item interactions, following He et al. (He et al., 2017). In the GMF part, users and items are embedded into a latent space, with element-wise multiplication capturing implicit interactions. In the MLP part, users and items are embedded and their concatenated embeddings pass through two linear layers with ReLU activations. The combined output from GMF and MLP is processed through a final linear layer and a sigmoid activation scaled appropriately to produce a prediction score. Despite extensive fine-tuning, this model performed poorly.

3.6. Ensemble

After observing that our different models have varying strengths, we adopted an ensemble approach for the final submission. After experimenting with different blending techniques, including gradient boosting, which we found to be less performant, we decided on using a Linear Regressor trained on the validation set used for training all previous models.

MODEL	PUBLIC SCORE
NCF	0.99909
SURPRISESVD (FINE-TUNED)	0.98954
SURPRISESVD++ (FINE-TUNED)	0.98959 TODO
SVD++ + LIGHTGBM	0.98137
ALS	0.98447
AUTOENCODER	0.98193
JNCF$_c$	0.98027
LIGHTGCN	0.98113
BMF	0.97946
BMF + REGRESSION + LIGHTGBM	0.97000
BMF + ORDERED PROBIT + LIGHTGBM	0.96777
ENSEMBLE	0.96677

Table 1. SCORES OF OUR MODELS

4. Results

Table 1 summarizes the performance of our models on the public dataset, excluding our experiments. All models were trained using 90% of the training set, with predictions on the validation set utilized to train the ensemble model. Notably, while BMF Ordered Probit (+LightGBM) demonstrates superior performance among the individual models, it is outperformed by the ensemble of all models. Notably, all our models performed better than the two chosen baselines.

5. Summary

In this report, we explored the most prominent approaches to Collaborative Filtering and rigorously tested their performance in the context of the Computational Intelligence Lab 2024 Kaggle competition. Our investigation included both traditional methods and modern innovations, providing a comprehensive overview of the field.

We began with classical methods such as matrix factorization to establish a baseline. We then explored advanced models, such as autoencoders and neural collaborative filtering, which capture complex, non-linear relationships in data. One notable experiment involved developing a model similar to J-NCF$_c$, using two Multilayer Perceptrons for embedding generation and concatenating these embeddings as input to a final Perceptron, resulting in significant improvements.

Recognizing that each model had unique strengths, we adopted an ensemble approach for our final submission. After testing various blending techniques, including gradient boosting, we found a linear regressor trained on the validation set to be most effective. This ensemble of all presented solutions achieved a score of 0.96677 in the competition.

Our work highlights the value of combining diverse methodologies to enhance CF performance, offering insights into their practical applications in data science environments.

References

Bennett, J., Lanning, S., et al. The netflix prize. In *Proceedings of KDD cup and workshop*, volume 2007, pp. 35. New York, 2007.

Chen, W., Cai, F., Chen, H., and Rijke, M. D. Joint neural collaborative filtering for recommender systems. *ACM Trans. Inf. Syst.*, 37(4), aug 2019. ISSN 1046-8188. doi: 10.1145/3343117. URL https://doi.org/10.1145/3343117.

Goldberg, D., Nichols, D., Oki, B. M., and Terry, D. Using collaborative filtering to weave an information tapestry. *Commun. ACM*, 35(12):61–70, dec 1992. ISSN 0001-0782. doi: 10.1145/138859.138867. URL https://doi.org/10.1145/138859.138867.

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., and Chua, T.-S. Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web*, pp. 173–182. International World Wide Web Conferences Steering Committee, 2017. URL https://arxiv.org/abs/1708.05031.

He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., and Wang, M. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '20, pp. 639–648, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450380164. doi: 10.1145/3397271.3401063. URL https://doi.org/10.1145/3397271.3401063.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. Lightgbm: a highly efficient gradient boosting decision tree. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, pp. 3149–3157, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.

Koren, Y. Factorization meets the neighborhood: a multifaceted collaborative filtering model. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '08, pp. 426–434, New York, NY, USA, 2008. Association for Computing Machinery. ISBN 9781605581934. doi: 10.1145/1401890.1401944. URL https://doi.org/10.1145/1401890.1401944.

Kuchaiev, O. and Ginsburg, B. Training deep autoencoders for collaborative filtering, 2017. URL https://arxiv.org/abs/1708.01715.

Liu, X. and Wang, Z. Cfda: Collaborative filtering with dual autoencoder for recommender system. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–7, 2022. doi: 10.1109/IJCNN55064.2022.9892705.

Rendle, S., Zhang, L., and Koren, Y. On the difficulty of evaluating baselines: A study on recommender systems, 2019. URL https://arxiv.org/abs/1905.01395.

Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., and Riedl, J. Grouplens: an open architecture for collaborative filtering of netnews. In *Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work*, CSCW '94, pp. 175–186, New York, NY, USA, 1994. Association for Computing Machinery. ISBN 0897916891. doi: 10.1145/192844.192905. URL https://doi.org/10.1145/192844.192905.

Sarwar, B., Karypis, G., Konstan, J., and Riedl, J. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web*, pp. 285–295, 2001.

Sedhain, S., Menon, A. K., Sanner, S., and Xie, L. Autorec: Autoencoders meet collaborative filtering. In *Proceedings of the 24th International Conference on World Wide Web*, WWW '15 Companion, pp. 111–112, New York, NY, USA, 2015. Association for Computing Machinery. ISBN 9781450334730. doi: 10.1145/2740908.2742726. URL https://doi.org/10.1145/2740908.2742726.

Smith, B. and Linden, G. Two decades of recommender systems at amazon.com. *IEEE Internet Computing*, 21 (3):12–18, 2017. doi: 10.1109/MIC.2017.72.

Strub, F. and Mary, J. Collaborative Filtering with Stacked Denoising AutoEncoders and Sparse Inputs. In *NIPS Workshop on Machine Learning for eCommerce*, Montreal, Canada, December 2015. URL https://inria.hal.science/hal-01256422.